



Terraform

Terraform

- Open Source
- Mozilla Public License 2.0
- Created by Hashicorp (vagrant, consul, packer, vault)
- Started in 2014
- Written in Go
- Pluggable

What is terraform?

Terraform original goal

Terraform is a tool to *Build, Change*, and *Version Control* your infrastructure.

Building infrastructure

- Talk to multiple cloud/infrastructure providers
- Ensure creation and consistency
- Express in an API-Agnostic DSL



Change infrastructure

- Apply incremental changes
- Destroy when needed
- Preview changes
- Scale easily

Version control

- HCL language
- State file (don't store it in your git repo)

Terraform concepts

Variables

- Terraform takes variables as input
- Typed variables
- JSON, HCL or CLI

Output

- Output specific text, ip addresses, ...
- Console or JSON

Terraform concepts

Providers

- Configuration of a resource provider
- e.g.: cloud, dns provider, ...
- Often require URL, API Key, ...

Modules

- Group of resources
- Takes parameters
- Returns output

Terraform concepts

Plan

- terraform plan
- Creates a plan of changes required
- Does nothing to the infra

Apply

- terraform apply
- Applies a plan, make all the changes
- Can make the plan before if needed

Terraform concepts

Statefile

- Current know state of the infra
- Stored in file or externally
- Locking

Destroy

- terraform destroy
- Delete all the resources
- resources can be "protected" in config



Install on Linux

Install on Linux

To install Terraform, find the appropriate package for your system and download it as a zip archive.

After downloading Terraform, unzip the package. Terraform runs as a single binary named `terraform`. Any other files in the package can be safely removed and Terraform will still function.

Finally, make sure that the `terraform` binary is available on your `PATH`. This process will differ depending on your operating system.

Install on Linux

Print a colon-separated list of locations in your `PATH`.

```
$ echo $PATH
```

Move the Terraform binary to one of the listed locations. This command assumes that the binary is currently in your downloads folder and that your `PATH` includes `/usr/local/bin`, but you can customize it if your locations are different.

```
$ mv ~/Downloads/terraform /usr/local/bin/
```

Verify the installation

Verify that the installation worked by opening a new terminal session and listing Terraform's available subcommands.

```
$ terraform -help
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.
## ...
```

Verify the installation

Add any subcommand to `terraform -help` to learn more about what it does and available options.

```
$ terraform -help plan
```

Troubleshoot

If you get an error that terraform could not be found, your `PATH` environment variable was not set up properly. Please go back and ensure that your `PATH` variable contains the directory where Terraform was installed.

Enable tab completion

If you use either bash or zsh you can enable tab completion for Terraform commands. To enable autocomplete, run the following command and then restart your shell.

```
$ terraform -install-autocomplete
```



Hello Terraform

Terraform introduction

TERRAFORM

```
provider "aws" {  
  // Set AWS_ACCESS_KEY_ID &  
  // AWS_SECRET_ACCESS_KEY env vars  
  region = "us-west-2"  
}  
  
resource "aws_s3_bucket" "my-cool-bucket" {  
  bucket = "my-cool-bucket"  
  acl = "public-read"  
}
```

The name to reference
within terraform files

The name of the bucket

CLOUD FORMATION

```
{  
  "Resources" : {  
    "my-cool-bucket" : {  
      "Type" : "AWS::S3::Bucket",  
      "Properties" : {  
        "AccessControl" : "PublicRead"  
      }  
    }  
  }  
}
```

Generated name:
stackname-my-cool-bucket-abc123id

TERRAFORM PLAN

MAIN.TF

```
# Configure aws with a default region
provider "aws" {
  region = "us-east-1"
}

# Create a demo s3 bucket
resource "aws_s3_bucket" "chadev_tf_demo" {
  bucket = "chadev-tf-demo"

  tags {
    Name = "Terraform demo for ChaDev"
    Environment = "Prod"
    TF = "yes"
  }
}
```

TERRAFORM PLAN

```
+ aws_s3_bucket.chadev_tf_demo
  acl: "" => "private"
  arn: "" => "<computed>"
  bucket: "" => "chadev-tf-demo"
  force_destroy: "" => "0"
  hosted_zone_id: "" => "<computed>"
  region: "" => "<computed>"
  tags.#: "" => "3"
  tags.Environment: "" => "Prod"
  tags.Name: "" => "Terraform demo for ChaDev"
  tags.TF: "" => "yes"
  website_domain: "" => "<computed>"
  website_endpoint: "" => "<computed>"
```

Plan: 1 to add, 0 to change, 0 to destroy.

TERRAFORM APPLY & STATE

TERRAFORM APPLY

```
aws_s3_bucket.chadev_tf_demo: Creating...
  acl:      "" => "private"
  arn:      "" => "<computed>"
  bucket:   "" => "chadev-tf-demo"
  force_destroy: "" => "0"
  hosted_zone_id: "" => "<computed>"
  region:   "" => "<computed>"
  tags.#:   "" => "3"
  tags.Environment: "" => "Prod"
  tags.Name: "" => "Terraform demo for ChaDev"
  tags.TF:   "" => "yes"
  website_domain: "" => "<computed>"
  website_endpoint: "" => "<computed>"
aws_s3_bucket.chadev_tf_demo: Creation complete

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

State path: terraform.tfstate
```

TERRAFORM STATE

```
{
  "version": 1,
  "serial": 1,
  "modules": [
    {
      "path": [
        "root"
      ],
      "outputs": {},
      "resources": {
        "aws_s3_bucket.chadev_tf_demo": {
          "type": "aws_s3_bucket",
          "primary": {
            "id": "chadev-tf-demo",
            "attributes": {
              "acl": "private",
              "arn": "arn:aws:s3:::chadev-tf-demo",
              "bucket": "chadev-tf-demo",
              "cors_rule.#": "0",
              "force_destroy": "false",
              "hosted_zone_id": "Z3AQBSTGFYJSTF",
              "id": "chadev-tf-demo",
              "policy": "",
              "region": "us-east-1",
              "tags.#": "3",
              "tags.Environment": "Prod",
              "tags.Name": "Terraform demo for ChaDev",
              "tags.TF": "yes",
              "website.#": "0"
            }
          }
        }
      }
    }
  ]
}
```

TERRAFORM STATE

- JSON
- Updated when Terraform runs
- Refreshed before an operation takes place
- Backwards compatible between TF versions

USING REMOTE STATE AS A DEPENDENCY

TANGENT: USING REMOTE STATE AS A DEPENDENCY

```
resource "terraform_remote_state" "core" {
  backend = "s3"
  config {
    bucket = "a-tf-state-bucket"
    key    = "hosted/core.json"
    region = "us-east-1"
  }
}

resource "aws_route53_record" "my-awesome-subdomain.yhat.com" {
  zone_id = "${terraform_remote_state.core.output.yhat-zone-id}"
  name    = "my-awesome-subdomain.yhat.com"
  type    = "A"
  ttl     = "60"
  records = [
    "${aws_eip.my-elastic-ip.public_ip}"
  ]
}
```



LET'S TERRAFORM A SERVER

CREATING A SERVER

```
# Create a key pair for our servers
resource "aws_key_pair" "chadev_demo" {
    key_name = "chadev_demo_key"
    public_key = "ssh-rsa ..."
}

# Create a demo server
resource "aws_instance" "chadev_demo_server" {
    # CentOS 7 (x86_64) with Updates HVM
    ami = "ami-6d1c2007"
    instance_type = "t2.medium"
    key_name = "${aws_key_pair.chadev_demo.key_name}"
    tags {
        Name = "ChaDev Demo Server"
        TF = "yes"
    }
}
```



Get the name of the key by referencing the output of the `aws_key_pair`. Notice we're using the variable name we define.

TERRAFORM PLAN

```
+ aws_instance.chadev_demo_server
```

```
...
```

```
+ aws_key_pair.chadev_demo
```

```
...
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
```


CREATING A SERVER

Error applying plan:

1 error(s) occurred:

```
* aws_instance.chadev_demo_server: Error launching source instance: VPCResourceNotSpecified: The specified instance type can only be used in a VPC. A subnet ID or network interface ID is required to carry out the request.  
   status code: 400, request id:
```

Terraform does not automatically rollback in the face of errors. Instead, your Terraform state file has been partially updated with any resources that successfully completed. Please address the error above and apply again to incrementally change your infrastructure.

```
Plan: 1 to add, 0 to change, 0 to destroy.
```



LET'S TERRAFORM A VPC

PARTS OF A VPC

- VPC definition (CIDR block)
- Subnets
- Internet gateway to allow our subnet to route to the internet
- Routing table to define how traffic is routed
- Main route table association to tell the vpc what route table to use by default

THE VPC

```
#####  
# VPC  
#####  
  
resource "aws_vpc" "chadev_demo" {  
  cidr_block = "10.20.0.0/16"  
  enable_dns_hostnames = true  
  
  tags {  
    Name = "ChaDev Demo VPC"  
    Env = "Prod"  
    TF = "yes"  
  }  
}
```

THE SUBNET

```
# Create a subnet for our instances
resource "aws_subnet" "chadev_demo_1a" {
  vpc_id = "${aws_vpc.chadev_demo.id}"
  cidr_block = "10.20.0.0/24"
  availability_zone = "us-east-1a"
  map_public_ip_on_launch = true

  tags {
    Name = "ChaDev Subnet AZ 1a"
    Env = "Prod"
    TF = "yes"
  }
}
```

THE INTERNET GATEWAY

```
# Create an internet gateway so we can talk to the world
resource "aws_internet_gateway" "main" {
  vpc_id = "${aws_vpc.chadev_demo.id}"

  tags {
    Name = "ChaDev VPC IGW"
    Env  = "Prod"
    TF   = "yes"
  }
}
```

THE ROUTE TABLE

```
# Replace the default routing table with one explicitly defined here
resource "aws_route_table" "chadev_demo" {
  vpc_id = "${aws_vpc.chadev_demo.id}"

  # Send all traffic not matched by other routing rules to the internet
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = "${aws_internet_gateway.main.id}"
  }

  tags {
    Name = "ChaDev Main RTB"
    Env = "Prod"
    TF = "yes"
  }
}
```

THE ROUTE TABLE ASSOCIATION

```
# Assign our route table as the default route table
resource "aws_main_route_table_association" "main" {
  vpc_id = "${aws_vpc.yhat-tf-demo.id}"
  route_table_id = "${aws_route_table.yhat-tf-default.id}"
}
```


ORGANIZING TERRAFORM FILES

- Terraform looks for all files ending in .tf
- I personally prefix the files with the provider where applicable
- Let's put the vpc in aws.vpc.tf
- Let's put the rest in aws.demo.tf

TERRAFORM APPLY

```
Error applying plan:
```

```
1 error(s) occurred:
```

```
* aws_instance.chadev_demo_server: Error launching source instance: VPCResourceNotSpecified: The specified instance type can only be used in a VPC. A subnet ID or network interface ID is required to carry out the request.  
   status code: 400, request id:
```

SPECIFY SUBNET AND SECURITY GROUPS

```
vpc_security_group_ids = [  
    "${aws_security_group.chadev_demo.id}",  
]  
  
subnet_id = "${aws_subnet.chadev_demo_1a.id}"
```

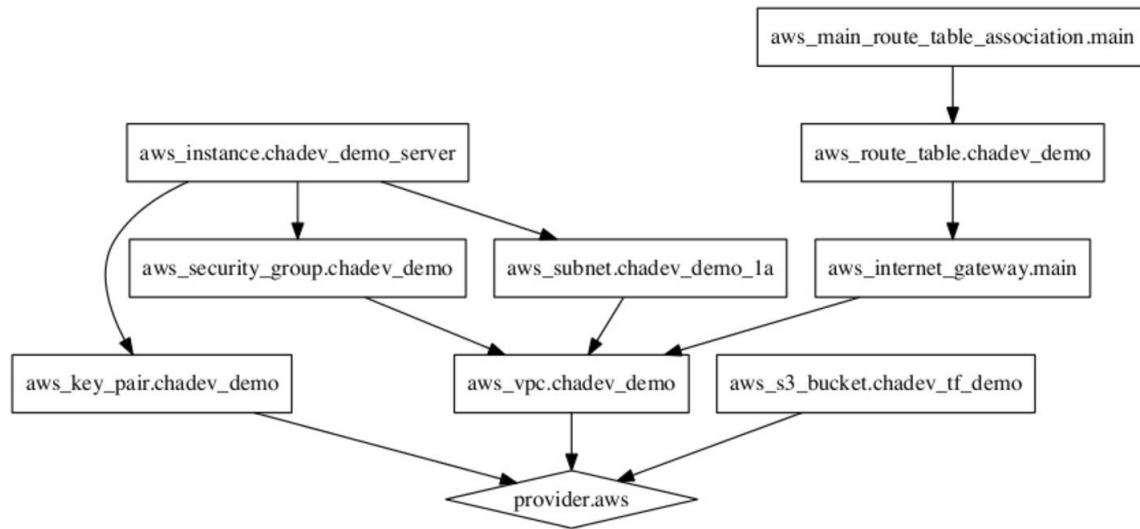


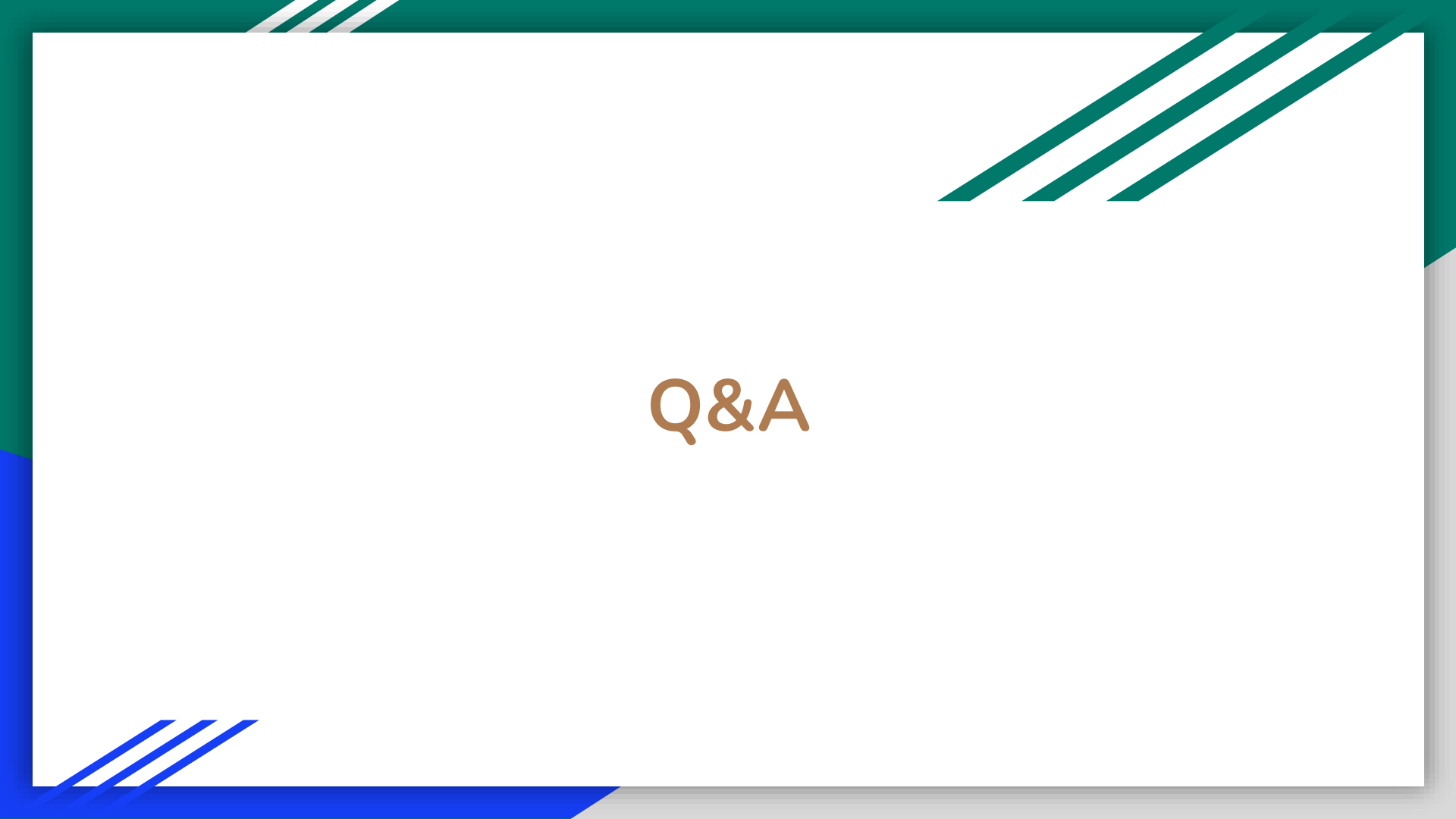
LAUNCH & REVIEW

TERRAFORM WILL GRAPH DEPENDENCIES

```
▶ terraform graph | dot -Tpng > graph.png
```

TERRAFORM GRAPH | DOT -TPDF > GRAPH.PDF





Q&A